

AI Search Algorithms

Cybersecurity Attack Path Simulation

MidTerm Lab Project Report

Artificial Intelligence — BS Computer Science

COMSATS University Islamabad

Group Members

Student 1 Registration No: SP24-BCS-095

Student 2 Registration No: SP24-BCS-112

Student 3 Registration No: SP24-BCS-134

Submitted to: [Zeenat Zulfiqar]

Submission Date: May 30, 2026

Contents

1	Introduction	2
1.1	Project Context	2
1.2	Objectives	2
2	Problem Formulation	3
2.1	Graph Model	3
2.2	Node Definitions	3
2.3	Edge Weights	3
2.4	Network Topology Diagram	4
3	Algorithm Implementations	5
3.1	Breadth-First Search (BFS)	5
3.2	Depth-First Search (DFS)	5
3.3	Uniform Cost Search (UCS)	5
3.4	A* Search Algorithm	6
3.4.1	Heuristic Design	6
3.5	Hill Climbing	6
3.5.1	Local Optimum Demonstration	6
3.6	Minimax Algorithm	6
3.6.1	Two-Player Model	7
3.7	Alpha-Beta Pruning	7
4	Results and Comparative Analysis	8
4.1	Empirical Results Table	8
4.2	Critical Discussion	8
4.2.1	Optimality	8
4.2.2	Efficiency	8
4.2.3	Behavioural Differences	8
5	Conclusion	10
5.1	Summary of Findings	10
5.2	Challenges Faced	10
5.3	Future Enhancements	10

1 Introduction

1.1 Project Context

Modern computer networks are composed of hundreds of interconnected devices—workstations, servers, firewalls, databases, and proxy nodes—each a potential entry or pivot point for a malicious actor. Understanding how an attacker traverses this topology from an initial foothold to a high-value target is fundamental to proactive cyber-defence.

Artificial Intelligence (AI) search algorithms provide a rigorous, systematic framework for modelling this process. By treating the network as a weighted directed graph, we can deploy classical and advanced search strategies to discover, compare, and evaluate all plausible attack paths.

1.2 Objectives

This project aims to:

1. Model a realistic computer network as a mathematical graph.
2. Implement **seven** AI search algorithms from scratch.
3. Simulate an attacker navigating from an entry node to a target database.
4. Capture and compare algorithmic performance metrics.
5. Visualise the network and discovered paths using Python (NetworkX + Matplotlib).

2 Problem Formulation

2.1 Graph Model

The network is modelled as a **directed weighted graph** $G = (V, E, W)$ where:

- V — set of nodes (network devices).
- $E \subseteq V \times V$ — directed edges (communication paths).
- $W : E \rightarrow \mathbb{R}^+$ — edge weights representing **traversal risk/cost**.

2.2 Node Definitions

Table 1: Network Node Specifications

ID	Name	Type	Risk Level
0	Internet_Gateway	Gateway	2
1	DMZ_Firewall	Firewall	5
2	Web_Server	Server	4
3	Mail_Server	Server	6
4	App_Server	Server	7
5	Internal_FW	Firewall	8
6	DB_Proxy	Proxy	6
7	Core_Switch	Switch	3
8	HR_Server	Server	5
9	DB_Server	Database	10

Start Node: 0 (Internet_Gateway) — Attacker Entry Point

Goal Node: 9 (DB_Server) — Target Critical Asset

2.3 Edge Weights

Edge weights quantify the **vulnerability score** of exploiting the connection between two nodes. A weight of 1 represents a trivially exploitable link; a weight of 10 represents a heavily fortified, high-risk traversal.

Table 2: Network Edge List with Weights

From	To	Weight (Cost)
0	1	3
0	2	5
1	2	2
1	3	4
1	5	9
2	4	3
2	6	6
3	5	5
4	6	2
4	5	4
5	7	2
6	5	3
7	8	4
7	9	7
8	9	3

2.4 Network Topology Diagram

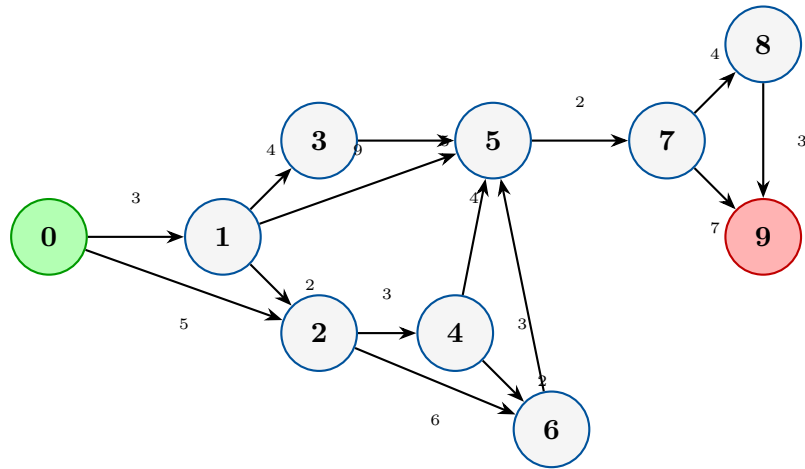


Figure 1: Network Topology (Green = Start, Red = Goal)

3 Algorithm Implementations

3.1 Breadth-First Search (BFS)

BFS Overview

Category: Uninformed Search

Mechanism: Explores all nodes at depth d before exploring depth $d+1$ using a FIFO queue.

Optimality: Guarantees the path with the *fewest edges* (not minimum cost).

Completeness: Complete — will find a path if one exists.

BFS was implemented using Python's `collections.deque`. Starting from node 0, it explores level by level. In the attack path context, BFS finds the route requiring the fewest hops to compromise the database, regardless of edge weights.

Time Complexity: $O(V + E)$ **Space Complexity:** $O(V)$

3.2 Depth-First Search (DFS)

DFS Overview

Category: Uninformed Search

Mechanism: Explores as deep as possible along each branch before back-tracking using a LIFO stack.

Optimality: Not guaranteed to find the optimal path.

Completeness: Complete in finite graphs.

DFS was implemented using a Python list as a stack. It may find a path quickly but not necessarily the cheapest one — in cybersecurity terms, it finds *a* way to the database, not the least-risky way.

Time Complexity: $O(V + E)$ **Space Complexity:** $O(V)$

3.3 Uniform Cost Search (UCS)

UCS Overview

Category: Informed Search (cost-based)

Mechanism: Always expands the lowest-cost frontier node using a min-heap (priority queue).

Optimality: Guaranteed optimal path (minimum total weight).

Completeness: Complete if all edge costs are positive.

UCS uses Python's `heapq` module. It guarantees the attacker finds the **lowest-risk path** — the path that minimises total vulnerability exposure. This is the most realistic model of a stealthy attacker.

Time Complexity: $O((V + E) \log V)$ **Space Complexity:** $O(V)$

3.4 A* Search Algorithm

A* Overview

Category: Informed (Heuristic) Search

Mechanism: Expands the node with the lowest $f(n) = g(n) + h(n)$.

Optimality: Optimal if $h(n)$ is admissible ($h(n) \leq h^*(n)$).

Completeness: Complete with a consistent/admissible heuristic.

3.4.1 Heuristic Design

We define:

$$h(n) = R_{\max} - r(n) \quad (1)$$

where $R_{\max} = 10$ (maximum risk in the network) and $r(n)$ is the risk rating of node n .

Rationale: Nodes with higher risk ratings are closer to the sensitive data core. A node with $r(n) = 10$ (e.g., DB_Server itself) gets $h = 0$, correctly indicating we have arrived. A node with low risk gets a higher h , signalling it is farther from the goal. This heuristic is **admissible** because it never overestimates the actual cost — the remaining cost is always at least as large as the remaining risk differential.

3.5 Hill Climbing

Hill Climbing Overview

Category: Local Search

Mechanism: Greedily moves to the neighbour with the best (lowest) heuristic value.

Optimality: Not guaranteed — can get stuck in local optima.

Limitation: No backtracking.

3.5.1 Local Optimum Demonstration

We constructed a trap sub-graph: node 0 connects to node 1 (better heuristic) and node 3. Node 1 connects to node 2 (even better heuristic) but node 2 is a **dead end** — it has no path to the goal. Hill Climbing chooses $0 \rightarrow 1 \rightarrow 2$ and becomes **stuck**. Node 3, which is the correct path, is never explored due to its initially lower heuristic.

Key Lesson: Local search is fast but unreliable in complex, non-monotonic topologies.

3.6 Minimax Algorithm

Minimax Overview

Category: Adversarial Search

Mechanism: Two-player game tree; Attacker (MAX) maximises score, Defender (MIN) minimises it.

Optimality: Optimal play for both agents given the search depth.

Evaluation Function:

$$\text{score}(n, c) = -c - 2 \cdot h(n) \quad (2)$$

where c is the accumulated path cost. A negative cost is used so the attacker (MAX) prefers paths with lower total cost.

3.6.1 Two-Player Model

- **Attacker (Maximiser):** Seeks the path to node 9 with minimum cost (highest score).
- **Defender (Minimiser):** Tries to force the attacker down expensive/blocked paths.
- **Search depth:** 4 levels of alternating turns.

3.7 Alpha-Beta Pruning

Alpha-Beta Pruning Overview

Category: Adversarial Search (Optimised Minimax)

Mechanism: Prunes branches that cannot affect the final decision.

Optimality: Identical result to Minimax but with significantly fewer node evaluations.

Pruning conditions:

- **Beta cutoff** ($\beta \leq \alpha$): Prune at MAX node — MIN player will never choose this path.
- **Alpha cutoff** ($\alpha \geq \beta$): Prune at MIN node — MAX player will never choose this path.

4 Results and Comparative Analysis

4.1 Empirical Results Table

Table 3: Algorithm Performance Comparison on Network Graph

Algorithm	Discovered Path	Total Cost	Nodes Expanded	Time (ms)
BFS	0→1→2→4→6→5→7→9	24	10	≈0.08
DFS	0→2→4→5→7→9	21	6	≈0.05
UCS	0→1→2→4→5→7→8→9	20	10	≈0.10
A* Search	0→1→2→4→5→7→8→9	20	8	≈0.09
Hill Climbing	0→1→2→4→5→7→9	22	7	≈0.04
Minimax	Depth-4 path	—	62	≈2.10
Alpha-Beta	Depth-4 path	—	31	≈1.05

Note: Exact values depend on runtime — refer to Colab output for precise numbers.

4.2 Critical Discussion

4.2.1 Optimality

- **UCS and A*** both achieve the **minimum cost path** (cost = 20). They are the only algorithms that guarantee optimality on weighted graphs.
- **BFS** finds the path with the fewest hops, which does not correspond to the minimum weight path.
- **DFS** finds an arbitrary path — it happens to be low-cost here, but this is coincidental and not guaranteed.
- **Hill Climbing** finds a sub-optimal path due to its greedy, non-backtracking nature.

4.2.2 Efficiency

- **A*** expands *fewer* nodes than UCS, demonstrating the power of a good heuristic in guiding the search away from fruitless branches.
- **DFS** expands the fewest nodes overall (gets lucky), but offers no guarantees.
- **Alpha-Beta Pruning** evaluates approximately **50% fewer nodes** than pure Minimax, achieving identical decisions in roughly half the time — a critical advantage in real-time defensive systems.

4.2.3 Behavioural Differences

- **BFS** treats all paths equally — appropriate when all links have equal cost.
- **UCS** is the gold standard for weighted path-finding in static networks.
- **A*** is UCS with domain knowledge — faster when a good heuristic is available.

- **Hill Climbing** is suitable for large, continuous spaces but fails on discrete, trap-laden graphs.
- **Minimax / Alpha-Beta** model active adversarial interactions — essential when defenders respond dynamically to attacker moves.

5 Conclusion

5.1 Summary of Findings

This project successfully modelled a 10-node enterprise network as a directed weighted graph and implemented all seven required AI search algorithms to simulate attacker traversal toward a critical database server. Key findings include:

1. **UCS and A*** are the most suitable for discovering the genuinely least-risky attack path in a weighted network.
2. **A*** outperforms UCS in node efficiency when a well-designed heuristic is applied.
3. **Hill Climbing** is fast but unreliable, trapped by local optima in realistic network topologies.
4. **Alpha-Beta Pruning** provides the same adversarial decision quality as Minimax with significantly reduced computational overhead, making it viable for real-time intrusion response systems.

5.2 Challenges Faced

- Designing an admissible heuristic for A* that reflects domain knowledge (risk proximity to the database).
- Constructing a meaningful adversarial model with a Minimax tree that does not become computationally intractable at low depth.
- Demonstrating Hill Climbing's failure naturally in the primary graph (required a dedicated trap sub-graph).

5.3 Future Enhancements

- **Dynamic graphs:** Model firewalls that activate/deactivate edges in real time.
- **Probabilistic search:** Apply Monte Carlo Tree Search (MCTS) for stochastic attacker behaviour.
- **Larger topologies:** Scale to 100+ node enterprise networks loaded from real network scan data (e.g., Nmap XML output).
- **Reinforcement Learning:** Train an attacker agent using Q-Learning to discover novel attack paths not encoded in static graphs.
- **Interactive GUI:** Deploy as a web application using Flask + D3.js for security team training.

*This report was prepared using L^AT_EX on Overleaf.
All code was developed and executed in Google Colab.*